

Principal Component Analysis (PCA)

Explanation, Examples & Other ML Techniques!

Julia Qiu

National Agroclimate Information Service

Idea Behind PCA

- Unsupervised learning
- “dimension reduction technique”
- Goal: reduce the number of “variables” (dimensions) to consider while retaining the most important information (*i.e. maximizing the “variance explained”*)
 - Imagine trying to response variable C using explanatory variable A and B
 - The more variance in A, the easier it is to determine B

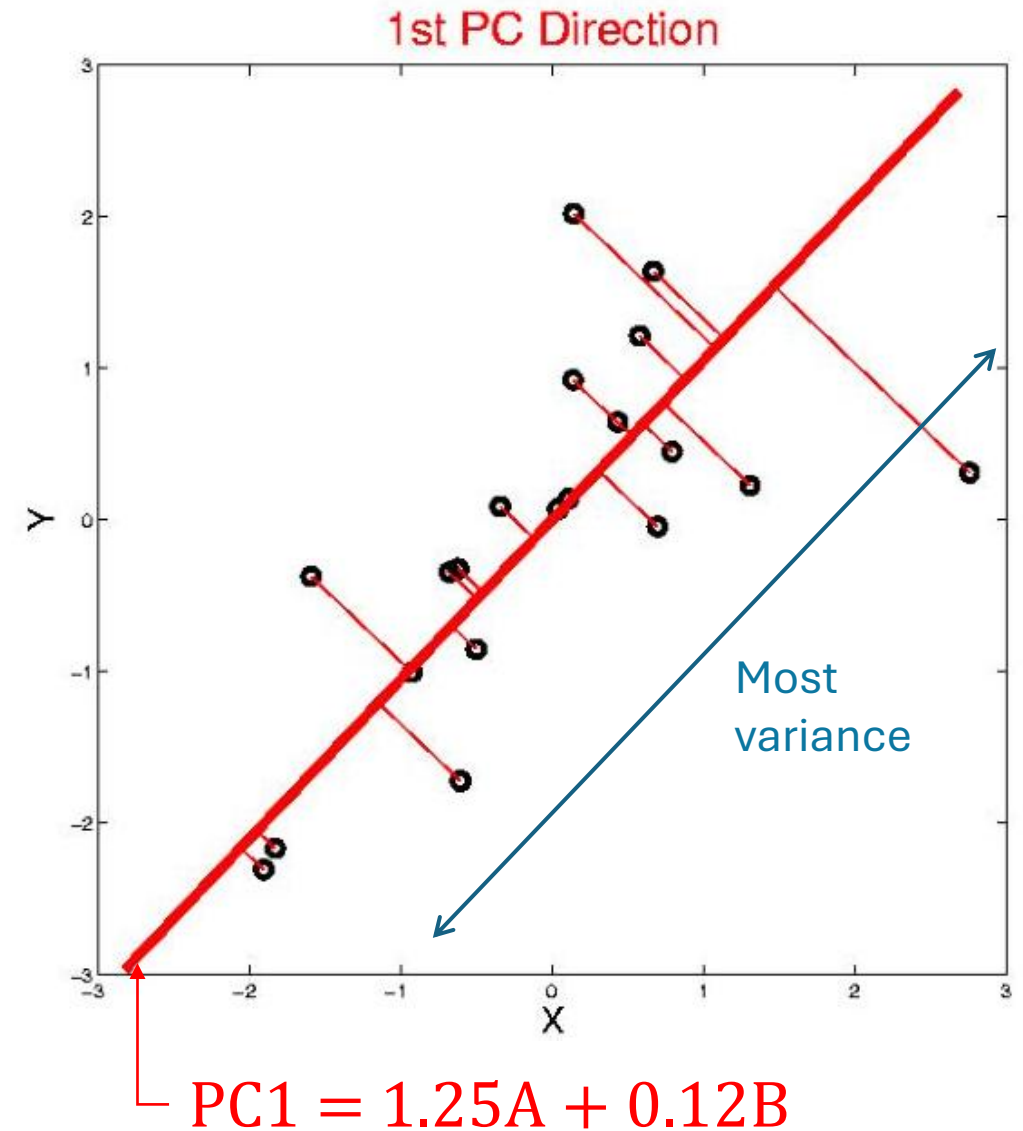
Explanatory Variable A	Explanatory Variable B	Response Variable C
1.23	110	0.5
4.69	111	1.5
8.94	108	2.5



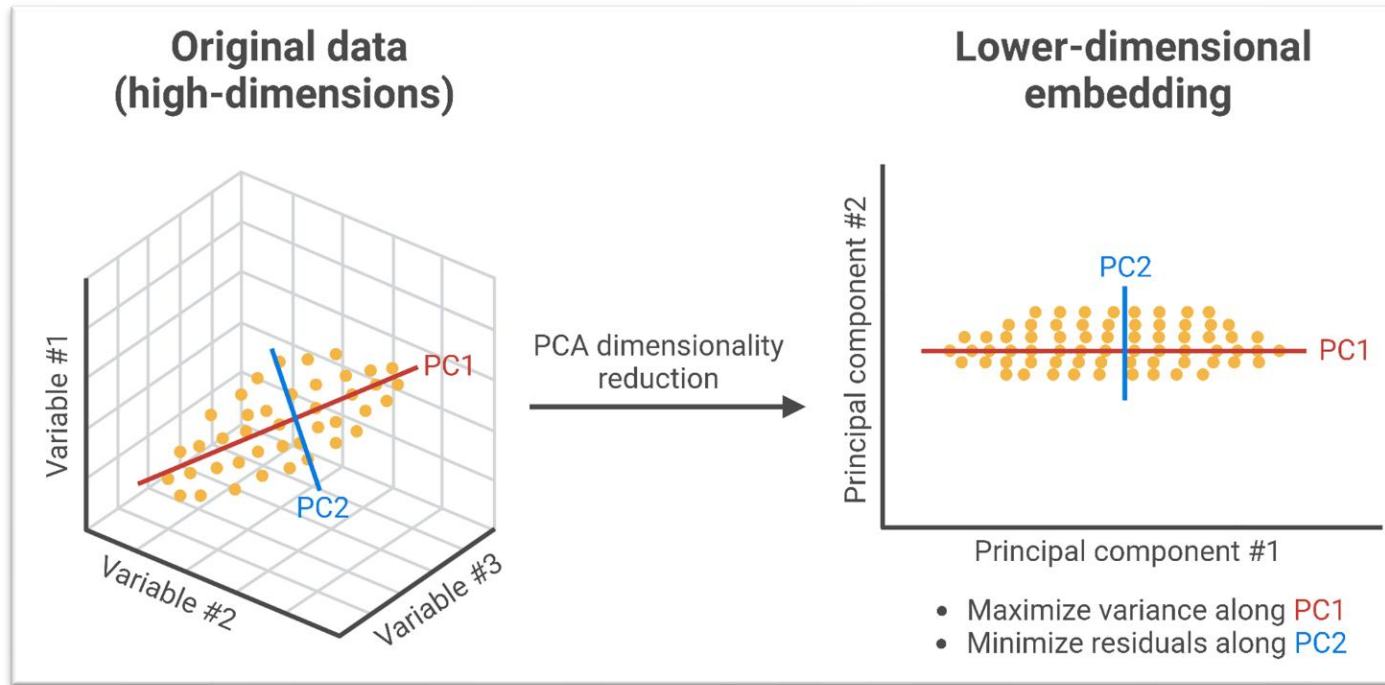
Explanatory Variable A	Explanatory Variable B	Response Variable C
12.42	107	?

Underlying Math

- We want to decrease the number of dimensions while also retaining the most information => use projections.
- **Project** the data from the variables with the **most explained variance** onto perpendicular vectors
 - are linear combinations of the original variables.
 - Minimize the residuals (distance between the data and the vector)= Principal Components (PCs)
- Hence, we can create as many PCs as there are variables.
- Then, we use the PCs that explain ~95% of variance.



More Underlying Math



[Principal Component Analysis \(PCA\) Transformation | BioRender Science Templates](#)

The PCs are made up of linear combinations of all the given variables.

- First, standardize the data.
- To minimize residuals (i.e. maximize explained variance), we use differentiation (Lagrange Multipliers).
- This simplifies to finding the eigenvalues/eigenvectors of the covariance matrix.
 - Eigenvectors = PCs
 - Amount of Explained Variance = Eigenvalue
- Choose the PCs (eigenvectors) with the highest eigenvalue (most explained variance).
- We have successfully reduced number of dimensions!

Pros and Cons

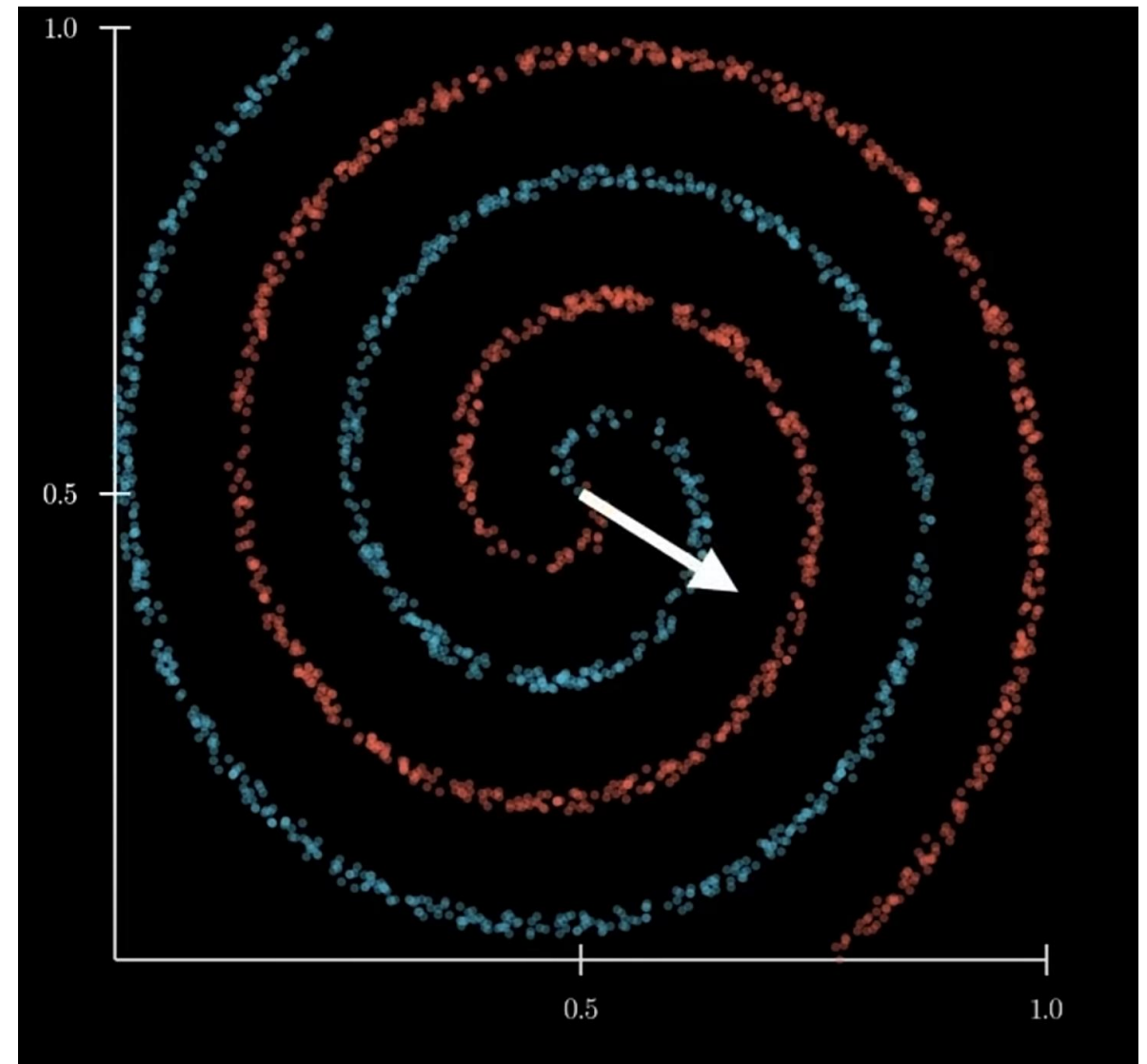
Pros

- Very fast
- Easy to understand output

Cons

- Does not work well with non-linear data

Consider: This technique may be too simple for our data!



[Latent Space Visualisation: PCA, t-SNE, UMAP | Deep Learning Animated](#)

Python Code Implementing PCA & Output

Note:

The following slides show Python code that implements transforming CDS training data using the PCA process and then using the PCs found during PCA as inputs to ML techniques.

The code and output shown hereafter is similar to what is shown in [demonstration.ipynb](#). However, [demonstration.ipynb](#) includes thorough explanation about the code in relation to the PCA and ML process, so it will probably be the most helpful to go through [demonstration.ipynb](#) (where you can run the code and then return here for more contextual information).

Python Script Implementing PCA & Output

- Imported Python Libraries:

```
import os
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.decomposition import PCA
from sklearn.compose import ColumnTransformer
import matplotlib.pyplot as plt
import plotly.express as px
import cds_utilities as cds
from sklearn.metrics import (
    classification_report,
    precision_score,
    recall_score,
    f1_score,
    cohen_kappa_score,
    confusion_matrix,
    accuracy_score
)
import seaborn as sns
from sklearn.pipeline import Pipeline
import time
```

- Beginning Code:

```
start = time.time()

# Potentially put these in cds_utilities.py
drought_labels = ['NoDrought', 'D0', 'D1', 'D2', 'D3/4']
drought_classifications = ['NoDrought', 'AbnormallyDry', 'ModerateDrought', 'SevereDrought', 'ExtremeDrought']
```

PCA Runtime: ~15 seconds

Overall Runtime (with LR < 1 minute and RF ~ 25 minutes): ~25-30 minutes

Python Script Implementing PCA & Output

- PCA Preprocessing:

```
# train_test_split is a utility from sklearn.model_selection that shuffles the data (by default)
# and splits it into 2 sides, the percentage size of the test portion is given by test_size
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=2)
```

✓ 1.7s

```
### Standardize the features
```

```
scaler = StandardScaler() # StandardScaler Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html#sklearn.preprocessing.StandardScaler
X_train_scaled = scaler.fit_transform(X_train) # Only fit to test data, since if we fit all X, the model would have information about the test information
X_test_scaled = scaler.transform(X_test)
```

✓ 1.3s

Python

- PCA (with 3 components):

```
# Choose the number of components based on your requirement
n_components = 3 # 3 components explains 0.6598615705836222 of cumulative variance
pca = PCA(n_components=n_components)
```

```
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)
```

✓ 0.4s

Python Script Implementing PCA & Output

- Variance code for PCA:

```

### Variance Values / Plots

# Explained Variance Ratio
explained_variance_ratio = pca.explained_variance_ratio_ # percentage of explained variance (in order of PC1, PC2, PC3,...)
print("Explained Variance Ratio:", explained_variance_ratio)

n_components = np.argmax(np.cumsum(explained_variance_ratio) >= 0.65) + 1 # +1 since array starts at 0
# ^ finds the first index where that running total reaches or exceeds 0.95.
print("Number of components to explain 65% of variance: ", n_components)

# Cumulative Expl. Variance
cumulative_explained_variance = explained_variance_ratio.cumsum() # returns ndarray but with each element as a sum of previous entries
print("Cumulative Expl. Variance Ratio:", cumulative_explained_variance)

# Explained Variance Ratio Plot
plt.plot(range(1, len(explained_variance_ratio)+1), explained_variance_ratio)
plt.xlabel('Number of Components')
plt.ylabel('Explained Variance')
plt.title('Explained Variance Ratio')
plt.show()

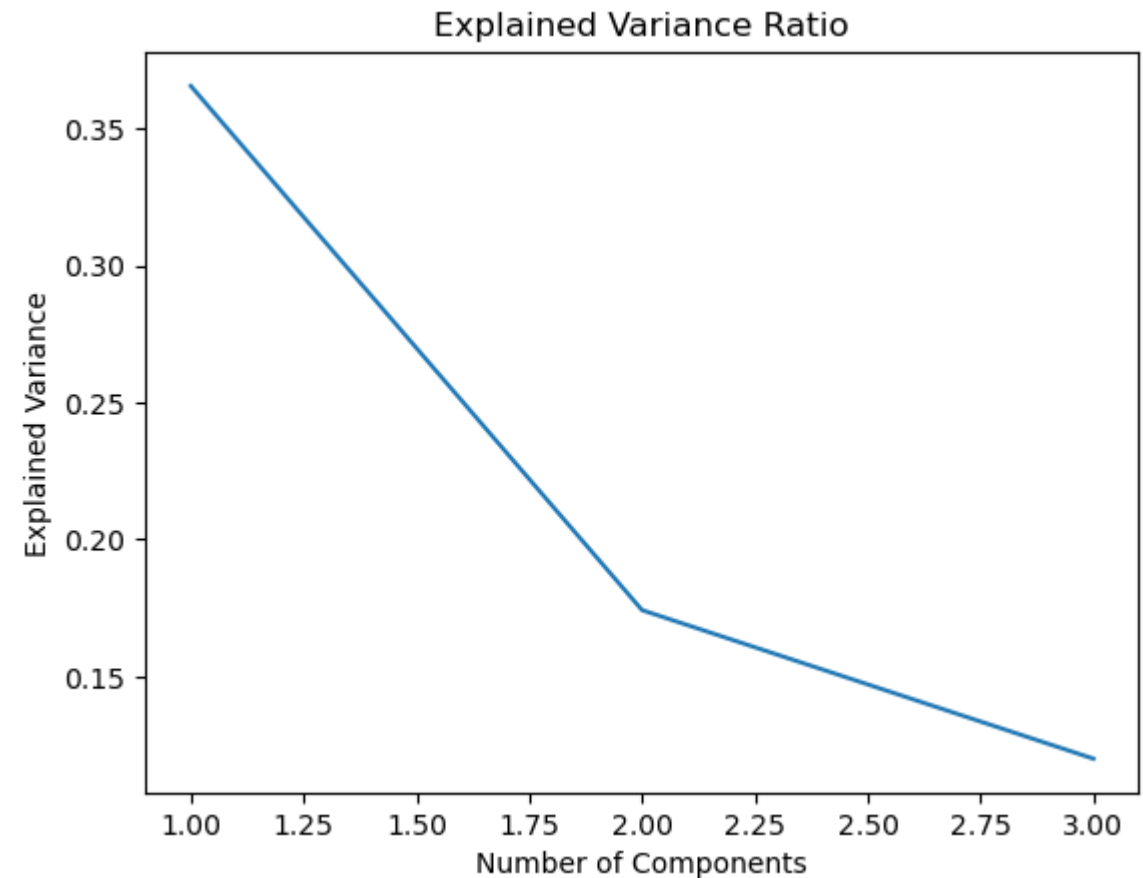
```

- Variance results from PCA:

```

Explained Variance Ratio: [0.36565414 0.1742242 0.11998323]
Number of components to explain 65% of variance: 3
Cumulative Expl. Variance Ratio: [0.36565414 0.53987834 0.65986157]

```

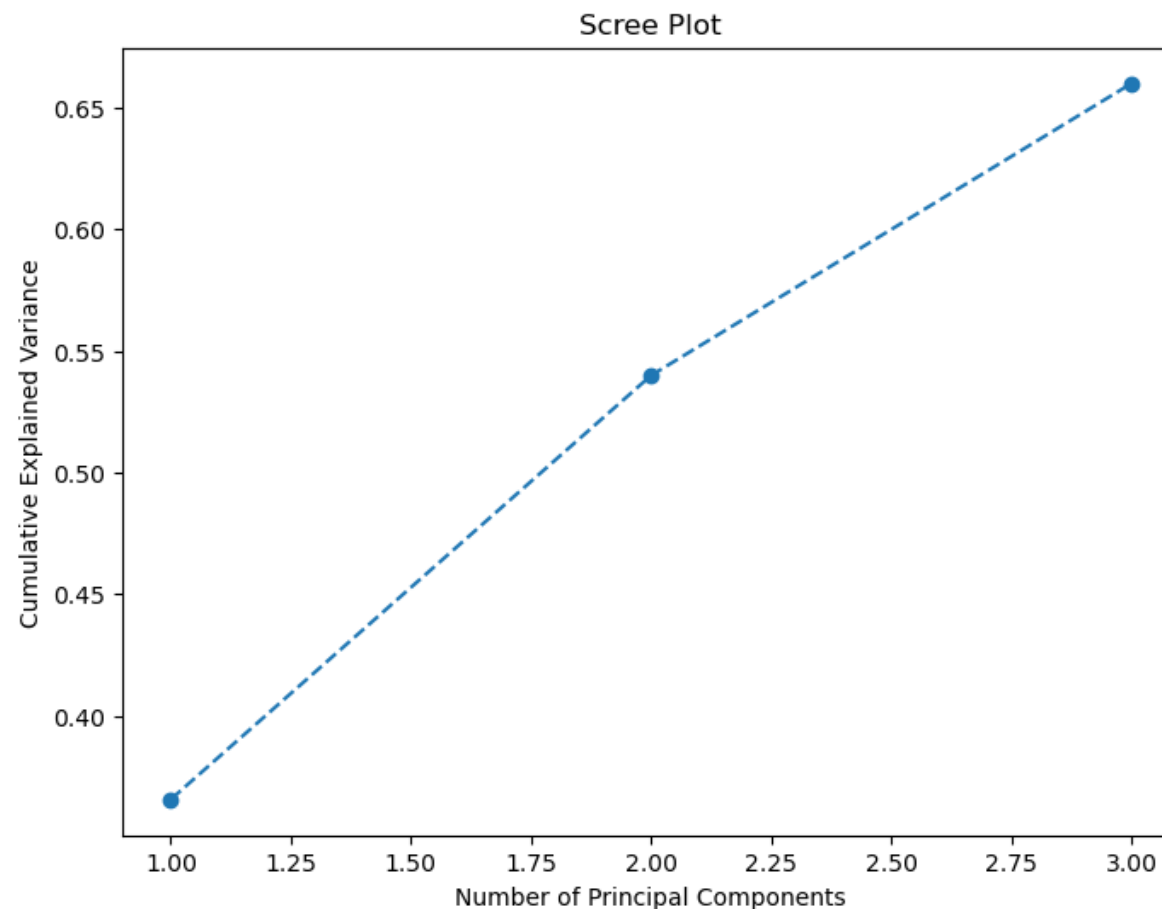


Python Script Implementing PCA & Output

- More variance code for PCA:

- More variance results from PCA:

```
# Scree Plot
plt.figure(figsize=(8, 6))
plt.plot(range(1, len(explained_variance_ratio) + 1), cumulative_explained_variance, marker='o', linestyle='--')
plt.title('Scree Plot')
plt.xlabel('Number of Principal Components')
plt.ylabel('Cumulative Explained Variance')
plt.show()
✓ 0.1s
```



```
total_var = pca.explained_variance_ratio_.sum() * 100
print("Total explained variance = ", total_var)
✓ 0.0s
```

Total explained variance = 65.98615705836222

Python Script Implementing PCA & Output

• PCA Results:

```
# Eigenvectors / Principal components
print("\nEigenvectors / Principal Components:")
print(pca.components_)
print(pca.components_.shape)

# Loading Matrix = matrix of eigenvectors
loading_matrix = pd.DataFrame(
    pca.components_.T, # transpose so we can see PCs as columns
    index=X.columns, # feature names are row names
    columns=[f'PC{i+1}' for i in range(pca.components_.shape[0])] # PC1, PC2, ...
)
print("\nLoading Matrix:")
print(loading_matrix)
print(loading_matrix.shape)
```

0.0s

```
Eigenvectors / Principal Components:
[[-0.00756775  0.01547402  0.2193832  0.26683318  0.28545525  0.29713266
   0.28988172  0.27342876  0.22730244  0.2705839  0.28721243  0.29434277
   0.28607604  0.26818425  0.14771011 -0.04352627 -0.04824493 -0.050548
  -0.04398452 -0.07237736 -0.10633677 -0.12499354 -0.08708637  0.1837134 ]
 [-0.09837476 -0.03628041  0.01392695 -0.0036026 -0.00435262  0.02193538
   0.04492389  0.03521394  0.00463979 -0.00928736 -0.00674416  0.02426953
   0.04547469  0.03872583  0.29414134  0.45400439  0.47877741  0.47825214
   0.47660512  0.00855642  0.00641003  0.01002656  0.03539411 -0.03897393]
 [ 0.09639117  0.08189563 -0.34282702 -0.27062218 -0.16527842  0.07869743
   0.17967056  0.2069448 -0.34382565 -0.26244502 -0.15265173  0.08967164
   0.18164969  0.20451562 -0.19357649  0.0485961  0.02617852  0.03196626
   0.02013369 -0.22583373 -0.30123472 -0.26848616 -0.11779253  0.34886655]]
(3, 24)
```

```
Loading Matrix:
      PC1      PC2      PC3
lat    -0.007568 -0.098375  0.096391
lon     0.015474 -0.036280  0.081896
spi_1m  0.219383  0.013927 -0.342827
spi_2m  0.266833 -0.003603 -0.270622
spi_3m  0.285455 -0.004353 -0.165278
spi_6m  0.297133  0.021935  0.078697
spi_9m  0.289882  0.044924  0.179671
...
MDProp -0.124994  0.010027 -0.268486
ADProp -0.087086  0.035394 -0.117793
NDProp  0.183713 -0.038974  0.348867
(24, 3)
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

Python Script Implementing PCA & Output

- PCA Results:

```
# Printing different versions of our data
print("X_train_scaled")
print(X_train_scaled)
print("X_train_scaled.shape =", X_train_scaled.shape)

print("\nX_train_pca")
print(X_train_pca)
print("X_train_pca.shape =", X_train_pca.shape)

✓ 0.0s
```

```
X_train_scaled
[[-0.88257852  0.94495531 -0.82834453 ... -0.5374175 -0.8199905
  0.99669696]
 [ 1.29384515 -0.38048879  0.0568345 ...  2.48553362  0.42799355
 -1.29095357]
 [-1.30655715 -0.40119885 -1.47034251 ...  2.21505274 -0.07610265
 -1.27244693]
 ...
 [ 0.38935739 -1.5195423 -0.16689207 ...  0.17515621  0.47187809
 -0.06394719]
 [ 0.21976594 -1.83019326  1.10737665 ... -0.5374175  0.42808594
  0.26830279]
 [ 1.01119273  1.17276601 -0.95479868 ... -0.5374175 -0.81662861
  0.99473492]]
X_train_scaled.shape = (2726630, 24)

X_train_pca
[[ 0.43788265  1.6678985  2.81640752]
 [-1.72716786  0.6318553 -1.65944372]
 [-4.85716262  1.72620488 -0.16803567]
 ...
 [ 1.03152607 -0.33157251 -0.34263237]
 [ 2.52168537 -0.80105539 -1.28454118]
 [ 1.38690555 -1.09625788  2.23347796]]
X_train_pca.shape = (2726630, 3)
```

Python Script Implementing PCA & Output

- PCA Results:

```
# Transformed Data as a pandas DataFrame (this is the same data as X_train_pca)
df_transformed = pd.DataFrame(X_train_pca, columns=[f'PC{i+1}' for i in range(X_train_pca.shape[1])])
print("Transformed Dataset:")
print(df_transformed)
```

✓ 0.0s

Transformed Dataset:

	PC1	PC2	PC3
0	0.437883	1.667898	2.816408
1	-1.727168	0.631855	-1.659444
2	-4.857163	1.726205	-0.168036
3	-7.629652	2.303943	0.805913
4	0.372299	-2.507384	-5.085070
...	...	...	...
2726625	-4.316190	-2.951524	0.114606
2726626	-0.001033	-2.843123	-0.214814
2726627	1.031526	-0.331573	-0.342632
2726628	2.521685	-0.801055	-1.284541
2726629	1.386906	-1.096258	2.233478

[2726630 rows x 3 columns]

Python Script Implementing PCA & Output

• PCA Results – 3D Representation of Transformed Data:

```

### Plot 3D representation of transformed data

# Randomly sample n points (can't plot 2.7 million)
n = 50000
np.random.seed(42) # seed = 42 for reproducibility
rand_idx = np.random.choice(len(X_train_pca), size = n, replace = False) # randomly choose n indices from the transformed data w/o replacement
# rand_idx is a np array of integers, which represents which rows (i.e data points) we will randomly select

X_sample = X_train_pca[rand_idx]
y_sample = y_train.iloc[rand_idx] # y_train is a pd df, so we can index by label/integer position,
# so .iloc (integer location) ensures we select the by integer position

fig = px.scatter_3d(
    x = X_sample[:,0],
    y = X_sample[:,1],
    z = X_sample[:,2],
    color = y_sample['cdm'],
    title = f'Total Explained Variance: {total_var:.2f}%',
    labels = {
        'x': 'PC1',
        'y': 'PC2',
        'z': 'PC3',
        'color': 'cdm'
    },
)
fig.show()

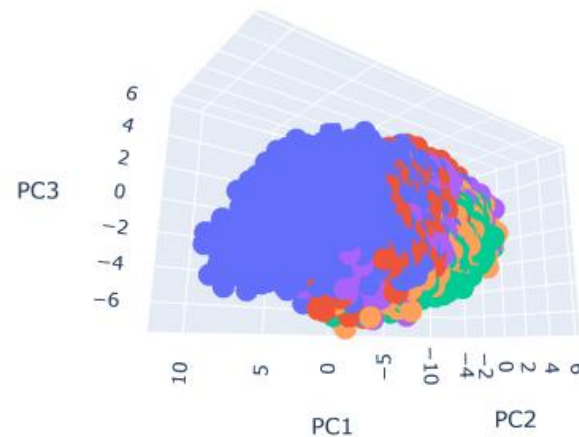
```

```

PCA_end = time.time() # since we finished PCA, so record when we finished
✓ 0.0s

```

Total Explained Variance: 65.99%



cdm

- NoDrought
- AbnormallyDry
- ExtremeDrought
- ModerateDrought
- SevereDrought

ML Classification Algorithms & Metrics

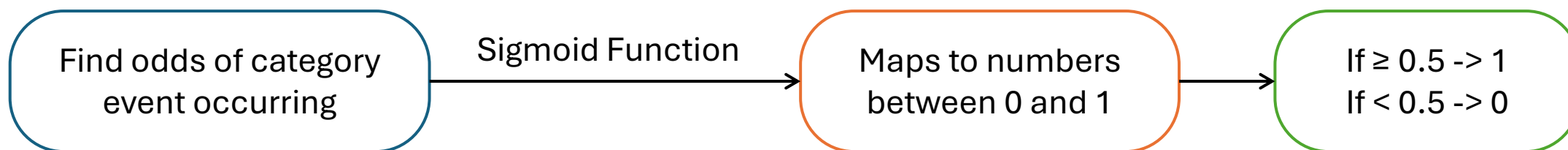
- Brief Introduction to Logistic Regression, Random Forest, and SVM
- ML Classification Metrics
- Evaluating Effectiveness of Using PCs in These ML Methods

Logistic Regression

[Logistic Regression in Machine Learning - GeeksforGeeks](#)

- Quick Summary:

- Relatively quick and simple supervised (response categories are assumed) classification method for binary categories (0 or 1).



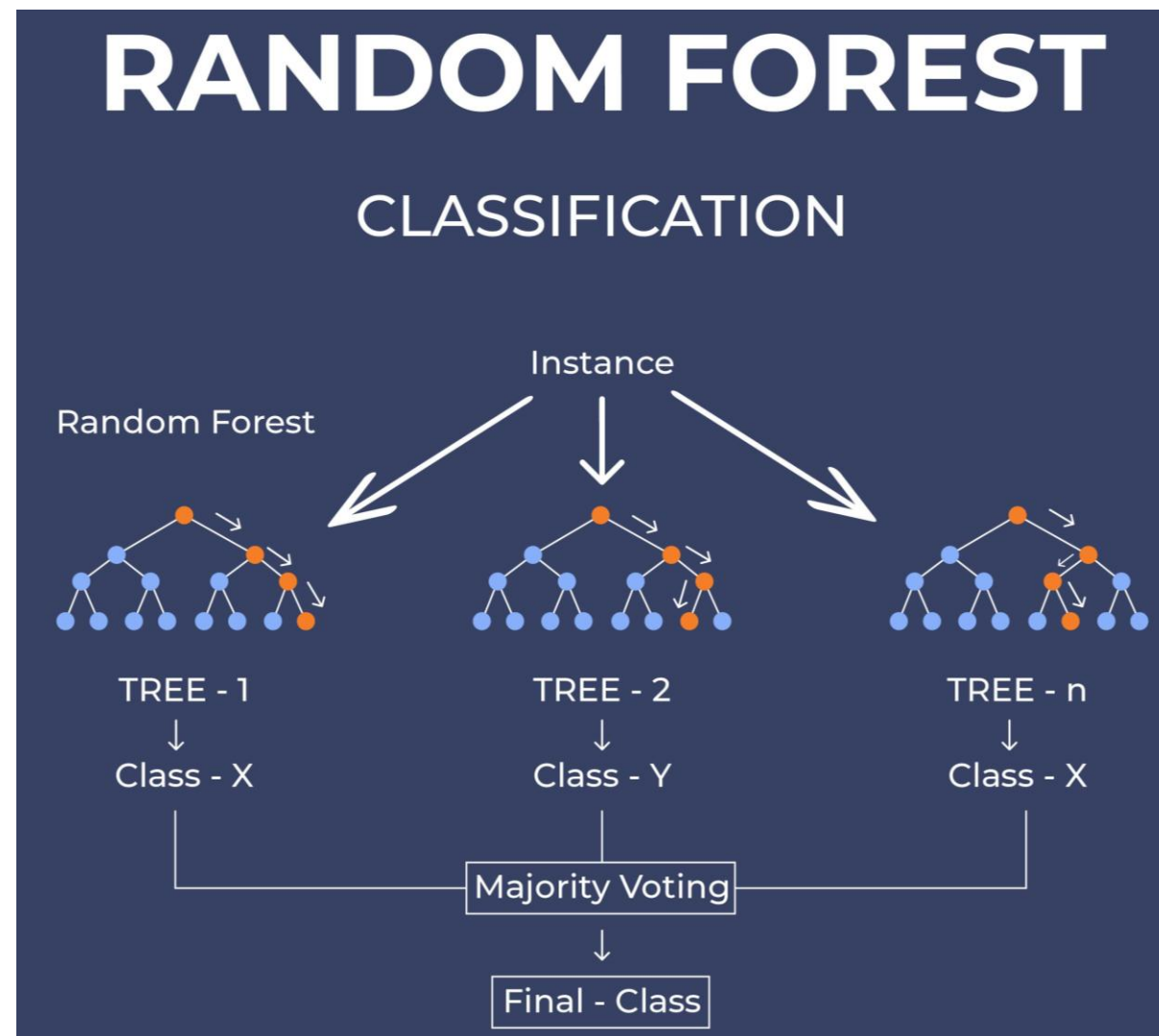
- Assumptions:

1. **Linear relationship** between the independent variables and the log odds of the dependent variable.
2. Response variable is **binary** (if not $\Rightarrow$ use *SoftMax* function instead of Sigmoid)
3. Independent observations (data points).
4. No extreme outliers.
5. Large sample size.

Random Forest

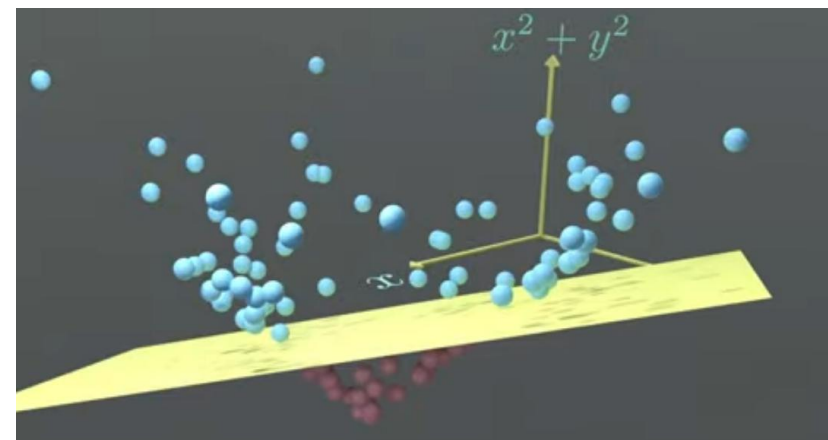
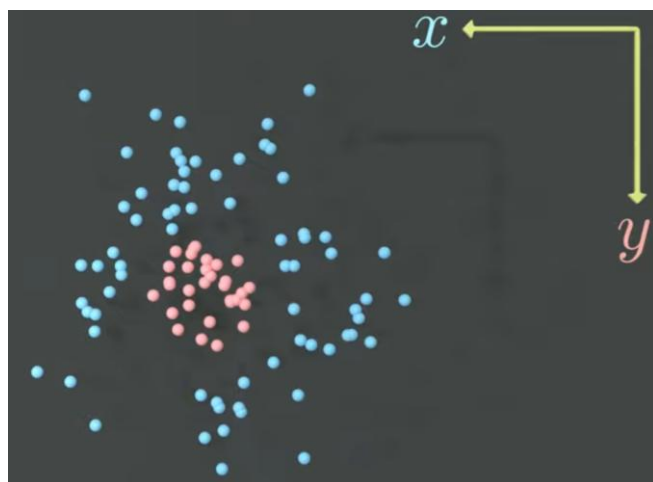
[Random Forest Algorithm in Machine Learning - GeeksforGeeks](#)

- Process:
 - Create multiple decision trees that consider a random sample of features and a random sample of data
 - Each tree is different!
 - Predicted classifications are made based on which category the majority of decision trees decide it is (Majority Voting).
- Good for avoiding overfitting!
- Assumption: Enough Data



Support Vector Machine (SVM)

- Supervised ML algorithm for classification and regression
- Main Purpose: “Find the best boundary (a.k.a hyperplane) that separates different classes” by maximizing distance between the hyperplane and data points



[Support Vector Machine \(SVM\) in 2 minutes](#)

- Failed Output: CDS data is too complex for planes to divide (20+ hours).

```
from sklearn.svm import SVC

svc = SVC(random_state=42)
svc.fit(X_train_pca, y_train)
y_pred_svc = svc.predict(X_test_pca)

print("SVM Accuracy:", accuracy_score(y_test, y_pred_svc))
print("Confusion matrix:\n", confusion_matrix(y_test, y_pred_svc))
```

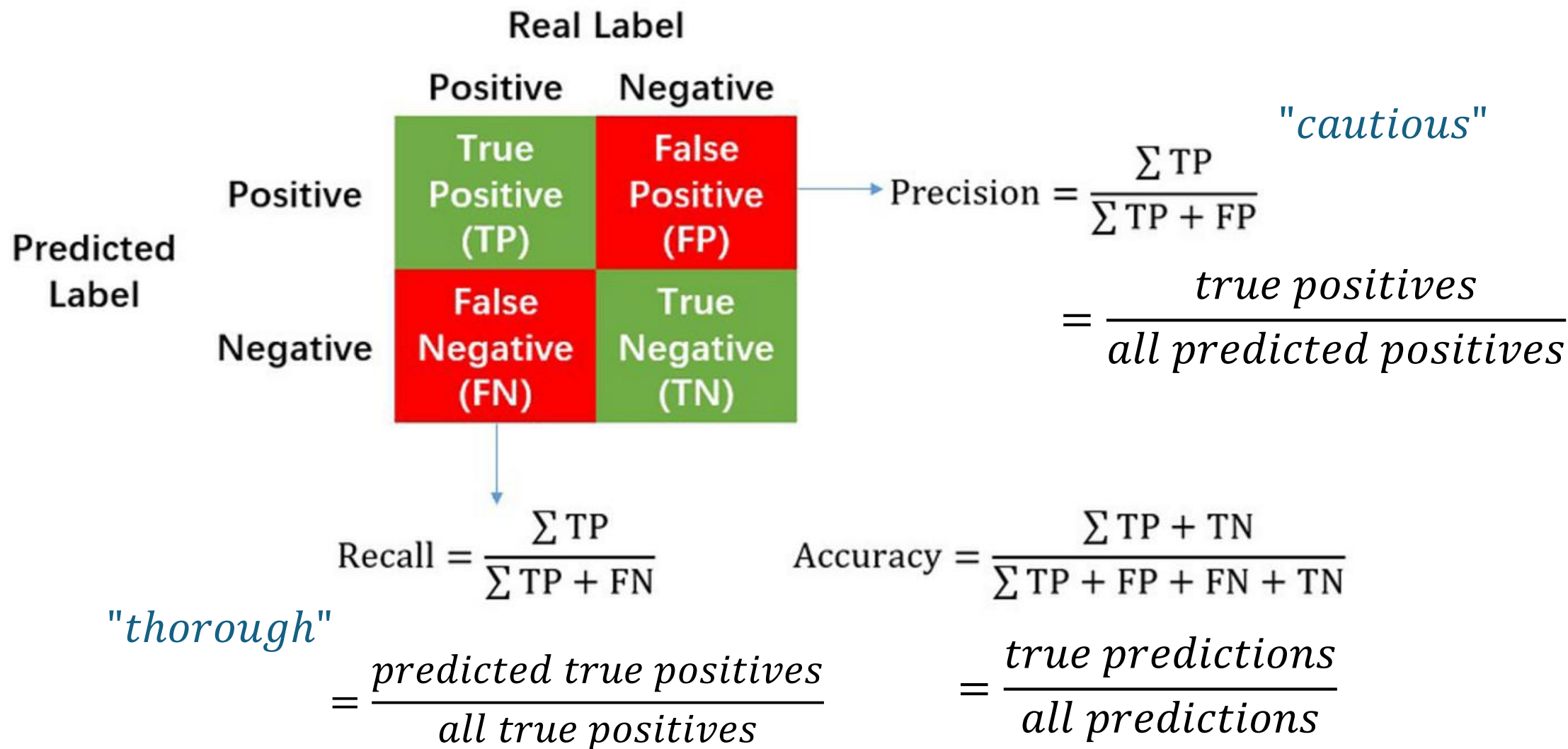
1204m 1.1s

Python

Discussing ML Algorithm Metrics

- Precision
- Recall
- Accuracy
- F1-score
- Micro vs Macro
- Cohen's Kappa
- Confusion matrix

ML Algorithm Metrics: Precision, Recall & Accuracy



ML Algorithm Metrics: F1-Score

- “harmonic mean of precision and recall”

$$F1 - score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

- Combines both precision and recall into one number, which is helpful if precision or recall are skewed.
- Balances being “cautious” (precision) and “thorough” (recall).
- The bigger F1-Score the better.

ML Algorithm Metrics: Micro vs Macro

Micro:

- Aggregates each class and then takes the average
- Good for category size imbalances

Macro:

- Calculates each category's ("class") average independently and then averages
- Treats all classes as equal

Ex. Find precision Pr with the given classes:

- Class A: 1 TP and 1 FP
- Class B: 1 TP and 1 FP
- Class C: 1 TP and 1 FP
- Class D: 10 TP and 90 FP

$$Pr(A) = Pr(B) = Pr(C) = 0.5, Pr(D) = 0.1$$

$$Pr_{micro} = \frac{1 + 1 + 1 + 10}{1 + 1 + 1 + 90} = 0.123$$

$$Pr_{macro} = \frac{0.5 + 0.5 + 0.5 + 0.1}{4} = 0.4$$

← More realistic since majority of data points are in class B

ML Algorithm Metrics: Cohen's Kappa

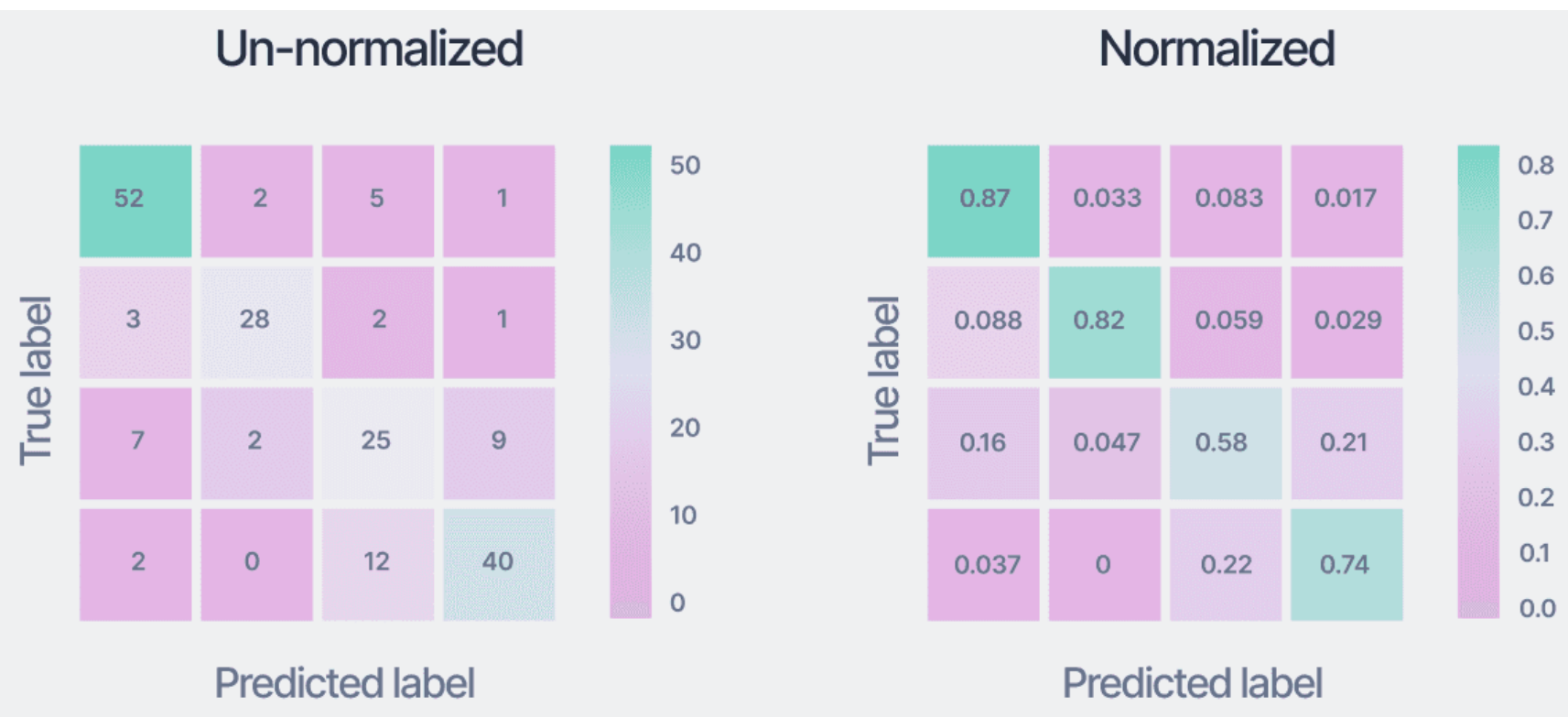
- The Cohen's Kappa statistic $\hat{k}$ measures the agreement between 2 category variables (in our case, true values and predicted values).
- The greater the $\hat{k}$ = the greater the agreement (i.e., the more our predictions match the actual values).
 - Values typically range from 0 to 1, but can be negative (rare).
- The formula is:

$$\hat{k} = \frac{p_0 - p_e}{1 - p_e}, \text{ where } p_0 = \text{the observed proportion of agreement (between the true and predicted values)}$$

p_e = the hypothetical probability of agreement by chance

ML Algorithm Metrics: Confusion matrix

- “Where does our model get confused?”
- Each column indicates the predicted category, and each row shows the actual category. The diagonal shows the predictions that were true.



- Showing the actual values isn't helpful with class imbalances
- Normalize the data to see what percentage of the class was predicted correctly
- We can normalize over the true (recall) or predicted (precision) values

Note: Consider `mord.LogisticAT()`
- Mord = multiclass ordinal data

Logistic Regression Metrics

```
from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(max_iter=500, random_state=42) # increase max_iter if convergence warning
lr.fit(X_train_pca, y_train)
y_pred_lr = lr.predict(X_test_pca) # our predicted values using Logistic Regression are stored in y_pred_lr
```

✓ 34.5s

```
print("Classification Report:")
print(classification_report(y_test, y_pred_lr, target_names=drought_classifications, labels=drought_classifications))
# pd.unique(y_test) creates a np ndarray of the unique drought strings as they appear in order
```

✓ 4.4s

```
Classification Report:
              precision    recall  f1-score   support

   NoDrought           0.80      0.94      0.86     429417
  AbnormallyDry        0.41      0.29      0.34     135729
 ModerateDrought       0.39      0.33      0.36      71351
  SevereDrought        0.40      0.25      0.31      33001
 ExtremeDrought        0.47      0.20      0.28      12160

 accuracy              0.70     681658
  macro avg           0.50      0.40      0.43     681658
  weighted avg       0.66      0.70      0.67     681658
```

Note: Consider `mord.LogisticAT()`
- Mord = multiclass ordinal data

Logistic Regression Metrics

```
print("Macro Precision:", precision_score(y_test, y_pred_lr, average='macro'))  
print("Macro Recall:", recall_score(y_test, y_pred_lr, average='macro'))  
print("Macro F1-score:", f1_score(y_test, y_pred_lr, average='macro'))  
  
print("\nMicro Precision:", precision_score(y_test, y_pred_lr, average='micro'))  
print("Micro Recall:", recall_score(y_test, y_pred_lr, average='micro'))  
print("Micro F1-score:", f1_score(y_test, y_pred_lr, average='micro'))
```

✓ 14.7s

Macro Precision: 0.4955723596372922

Macro Recall: 0.3998459001715182

Macro F1-score: 0.42907047102885787

Micro Precision: 0.6965032318259303

Micro Recall: 0.6965032318259303

Micro F1-score: 0.6965032318259303

Logistic Regression Metrics

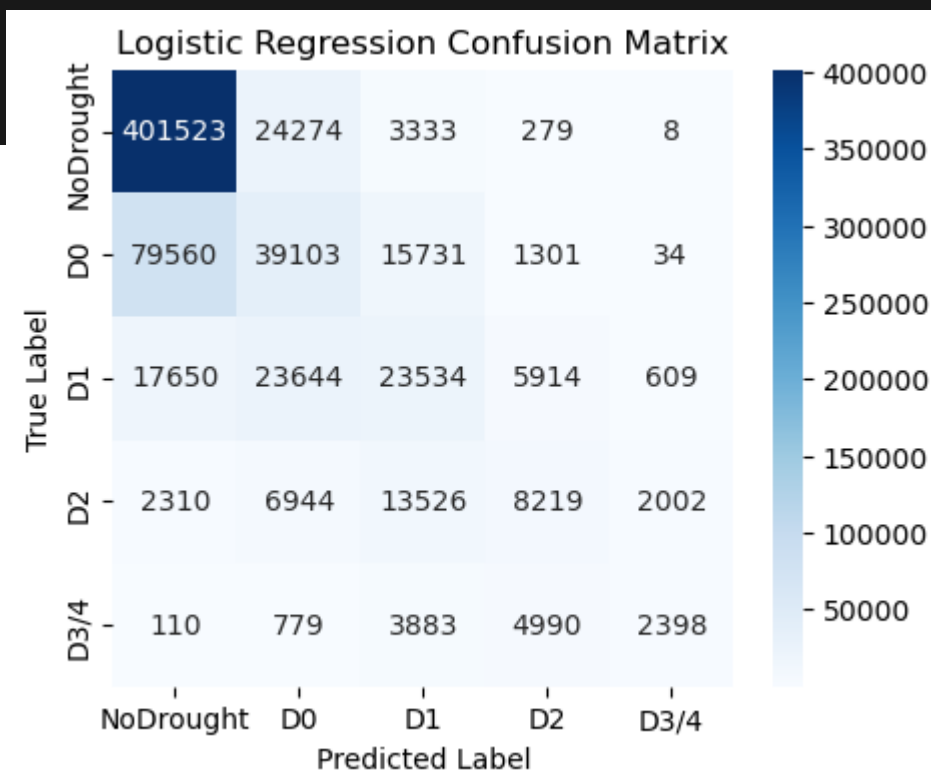
Note: Consider `mord.LogisticAT()`
- Mord = multiclass ordinal data

```
print("Cohen's Kappa:", cohen_kappa_score(y_test, y_pred_lr))
print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_lr))

cm_lr = confusion_matrix(y_test, y_pred_lr, labels=drought_classifications)
print("Logistic Regression Confusion matrix:\n", cm_lr)

plt.figure(figsize=(5,4))
sns.heatmap(cm_lr, annot=True, fmt='d', cmap='Blues', xticklabels=drought_classifications, yticklabels=drought_classifications)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Logistic Regression Confusion Matrix')
plt.show()
```

```
Cohen's Kappa: 0.39101299017766233
Logistic Regression Accuracy: 0.6965032318259303
Logistic Regression Confusion matrix:
[[401523  24274   3333    279     8]
 [ 79560  39103  15731   1301    34]
 [ 17650  23644  23534   5914   609]
 [  2310   6944  13526   8219  2002]
 [   110    779   3883   4990  2398]]
```



Logistic Regression Metrics

Note: Consider `mord.LogisticAT()`
- Mord = multiclass ordinal data

```
cm_lr_r = confusion_matrix(y_test, y_pred_lr, labels=drought_classifications, normalize='true')

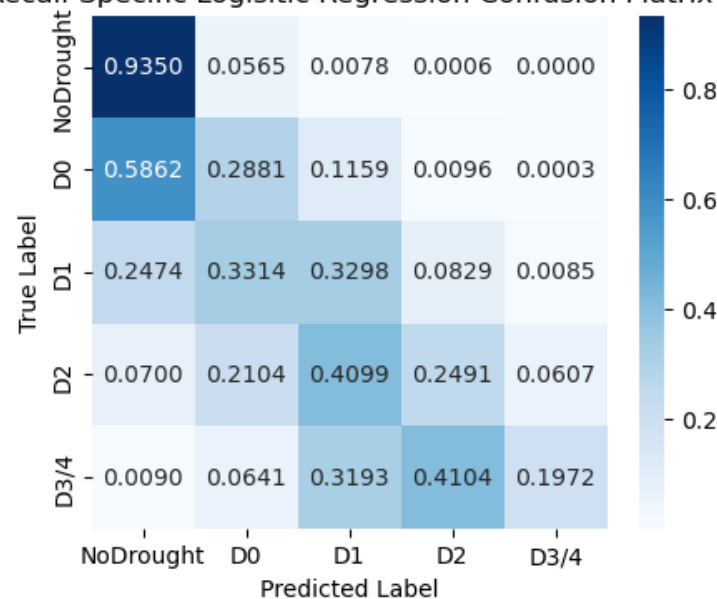
plt.figure(figsize=(5,4))
sns.heatmap(cm_lr_r, annot=True, fmt='.4f', cmap='Blues', xticklabels=drought_labels, yticklabels=drought_labels)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Recall-Specific Logistic Regression Confusion Matrix')
plt.show()

cm_lr_p = confusion_matrix(y_test, y_pred_lr, labels=drought_classifications, normalize='pred')

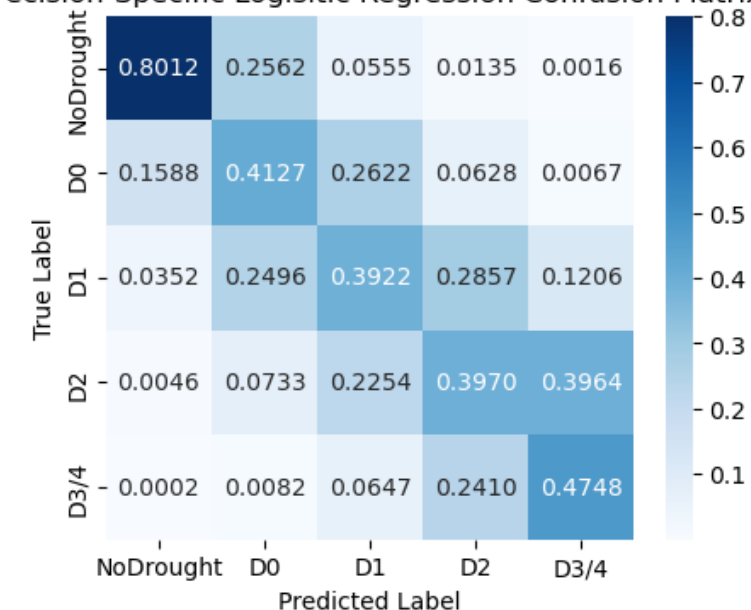
plt.figure(figsize=(5,4))
sns.heatmap(cm_lr_p, annot=True, fmt='.4f', cmap='Blues', xticklabels=drought_labels, yticklabels=drought_labels)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Precision-Specific Logistic Regression Confusion Matrix')
plt.show()
```

✓ 34s

Recall-Specific Logistic Regression Confusion Matrix



Precision-Specific Logistic Regression Confusion Matrix



Random Forest Metrics

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf = RandomForestClassifier(random_state=42)
```

```
rf.fit(X_train_pca, y_train)
```

```
y_pred_rf = rf.predict(X_test_pca)
```

✓ 21m 10.1s

```
print("Classification Report:")
```

```
print(classification_report(y_test, y_pred_rf, target_names=drought_classifications, labels=drought_classifications))
```

✓ 4.1s

Classification Report:

	precision	recall	f1-score	support
NoDrought	0.82	0.92	0.87	429417
AbnormallyDry	0.47	0.37	0.41	135729
ModerateDrought	0.47	0.41	0.44	71351
SevereDrought	0.49	0.38	0.43	33001
ExtremeDrought	0.59	0.47	0.53	12160
accuracy			0.72	681658
macro avg	0.57	0.51	0.53	681658
weighted avg	0.70	0.72	0.70	681658

Random Forest Metrics

```
print("Macro Precision:", precision_score(y_test, y_pred_rf, average='macro'))  
print("Macro Recall:", recall_score(y_test, y_pred_rf, average='macro'))  
print("Macro F1-score:", f1_score(y_test, y_pred_rf, average='macro'))  
  
print("\nMicro Precision:", precision_score(y_test, y_pred_rf, average='micro'))  
print("Micro Recall:", recall_score(y_test, y_pred_rf, average='micro'))  
print("Micro F1-score:", f1_score(y_test, y_pred_rf, average='micro'))
```

✓ 13.8s

```
Macro Precision: 0.569987571108221  
Macro Recall: 0.5089268154615981  
Macro F1-score: 0.5344398522123055
```

```
Micro Precision: 0.720176393440699  
Micro Recall: 0.720176393440699  
Micro F1-score: 0.720176393440699
```

Random Forest Metrics

```
print("Cohen's Kappa:", cohen_kappa_score(y_test, y_pred_rf))
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))

cm_rf = confusion_matrix(y_test, y_pred_rf, labels=drought_classifications)
print("Random Forest Confusion matrix:\n", cm_rf)

plt.figure(figsize=(5,4))
sns.heatmap(cm_rf, annot=True, fmt='d', cmap='Blues', xticklabels=drought_labels, yticklabels=drought_labels)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Random Forest Confusion Matrix')
plt.show()
```

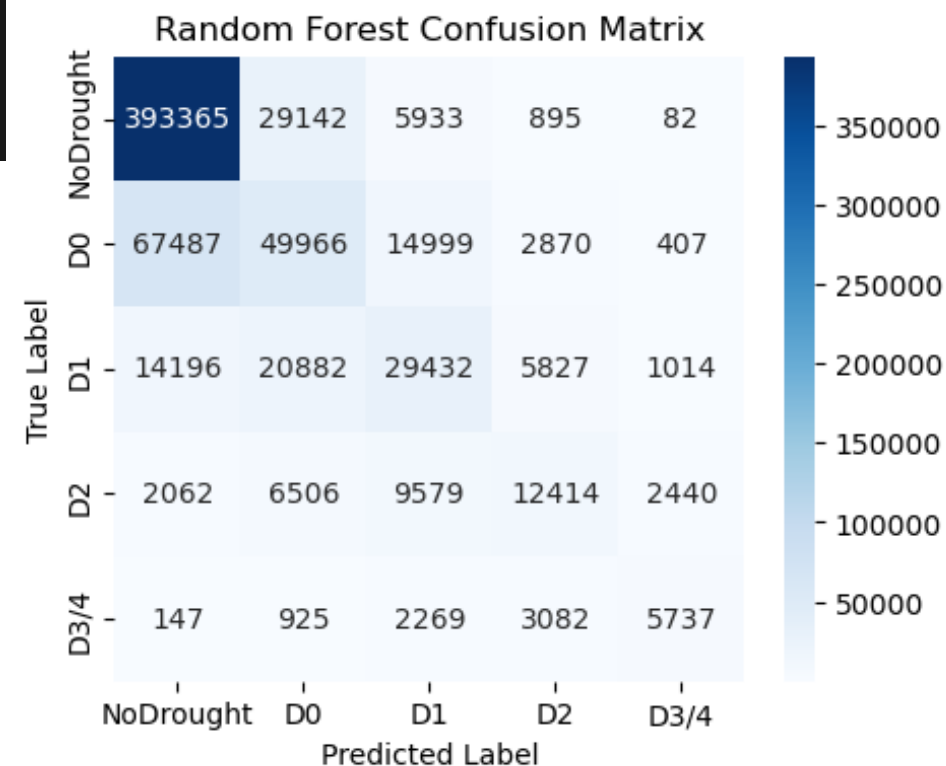
✓ 3.5s

Cohen's Kappa: 0.4576775803279486

Random Forest Accuracy: 0.720176393440699

Random Forest Confusion matrix:

```
[[393365  29142   5933    895    82]
 [ 67487  49966  14999   2870   407]
 [ 14196  20882  29432   5827  1014]
 [  2062   6506   9579  12414  2440]
 [   147    925   2269   3082  5737]]
```



Random Forest Metrics

```
cm_rf_r = confusion_matrix(y_test, y_pred_rf, labels=drought_classifications, normalize='true')

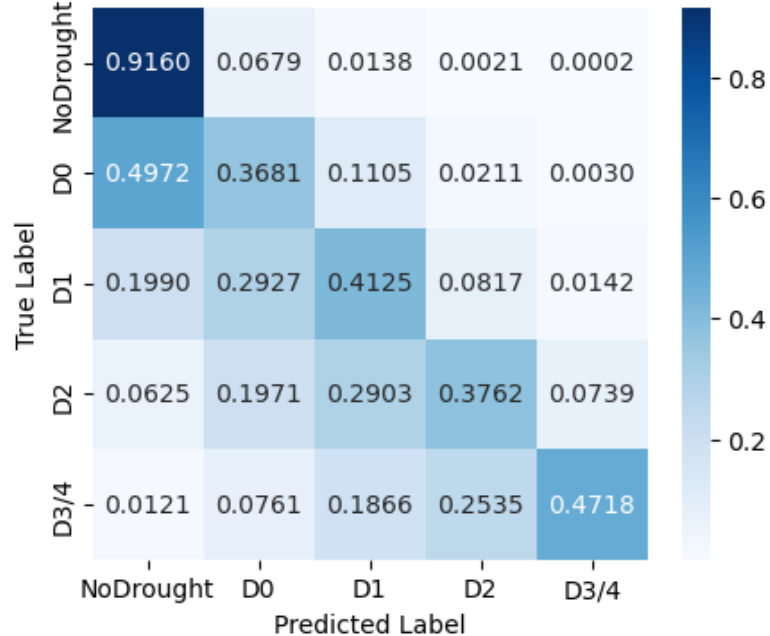
plt.figure(figsize=(5,4))
sns.heatmap(cm_rf_r, annot=True, fmt='.4f', cmap='Blues', xticklabels=drought_labels, yticklabels=drought_labels)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Recall-Specific Random Forest Confusion Matrix')
plt.show()

cm_rf_p = confusion_matrix(y_test, y_pred_rf, labels=drought_classifications, normalize='pred')

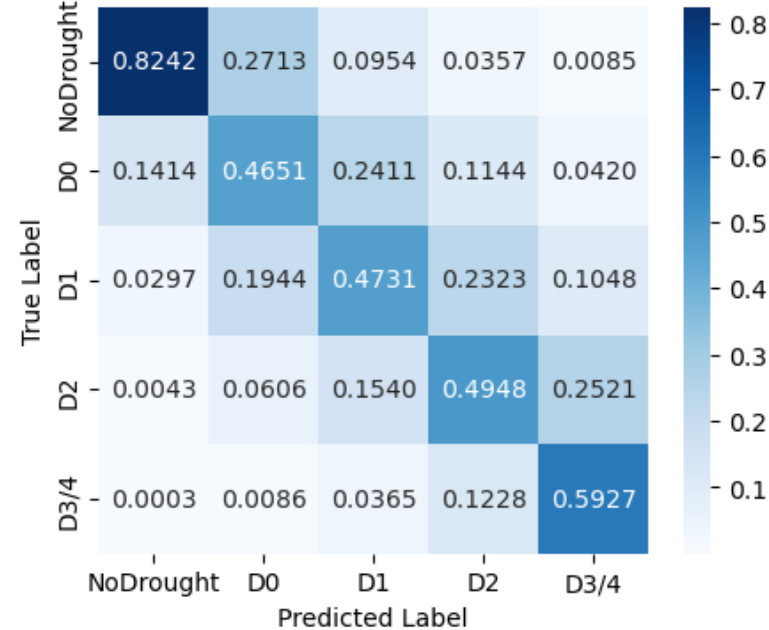
plt.figure(figsize=(5,4))
sns.heatmap(cm_rf_p, annot=True, fmt='.4f', cmap='Blues', xticklabels=drought_labels, yticklabels=drought_labels)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Precision-Specific Random Forest Confusion Matrix')
plt.show()
```

✓ 3.1s

Recall-Specific Random Forest Confusion Matrix



Precision-Specific Random Forest Confusion Matrix



Pipelining PCA and ML Algorithms

```

### Pipeline

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('pca', PCA(n_components=3)),
    ('classifier', LogisticRegression())
])

pipeline.fit(X_train_pca, y_train)
y_pred_pipe = pipeline.predict(X_test_pca)

print("Classification Report:")
print(classification_report(y_test, y_pred_pipe, target_names=drought_classifications)) # Or use your label list

print("Macro F1:", f1_score(y_test, y_pred_pipe, average='macro'))
print("Macro Precision:", precision_score(y_test, y_pred_pipe, average='macro'))
print("Macro Recall:", recall_score(y_test, y_pred_pipe, average='macro'))

print("\nMicro Precision:", precision_score(y_test, y_pred_pipe, average='micro'))
print("Micro Recall:", recall_score(y_test, y_pred_pipe, average='micro'))
print("Micro F1-score:", f1_score(y_test, y_pred_pipe, average='micro'))

print("Cohen's Kappa:", cohen_kappa_score(y_test, y_pred_pipe))
pl_lr_accuracy = pipeline.score(X_test_pca, y_test)
print(f"Pipeline Logistic Regression Accuracy: {pl_lr_accuracy}")

cm_pipe = confusion_matrix(y_test, y_pred_pipe, labels=drought_classifications, normalize='true')
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(5,4))
sns.heatmap(cm_pipe, annot=True, fmt='.4f', cmap="Blues", xticklabels=drought_labels, yticklabels=drought_labels)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Pipeline Confusion Matrix')
plt.show()

```

✓ 53.1s

Classification Report:

	precision	recall	f1-score	support
NoDrought	0.41	0.29	0.34	135729
AbnormallyDry	0.48	0.20	0.28	12160
ModerateDrought	0.39	0.33	0.36	71351
SevereDrought	0.80	0.94	0.86	429417
ExtremeDrought	0.40	0.25	0.31	33001
accuracy			0.70	681658
macro avg	0.50	0.40	0.43	681658
weighted avg	0.66	0.70	0.67	681658

Macro F1: 0.4288538930003389

Macro Precision: 0.4956325925298938

Macro Recall: 0.39966955618787403

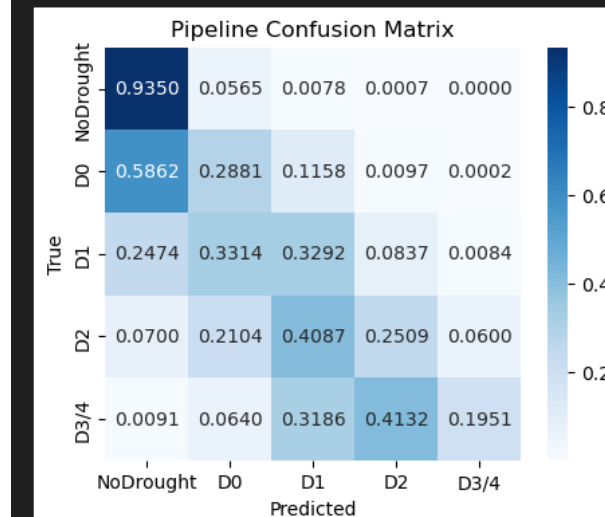
Micro Precision: 0.6964797596448659

Micro Recall: 0.6964797596448659

Micro F1-score: 0.6964797596448659

Cohen's Kappa: 0.39097468679254543

Pipeline Logistic Regression Accuracy: 0.6964797596448659



Runtime

```
total_end = time.time()
print(f"[PCA RUNTIME] {PCA_end - start} secs // {(PCA_end-start)/60.0} mins")
print(f"[OVERALL RUNTIME] {total_end - start} secs // {(total_end-start)/60.0} mins")
✓ 0.0s
```

```
[PCA RUNTIME] 30.419153690338135 secs // 0.5069858948389689 mins
[OVERALL RUNTIME] 1684.0391566753387 secs // 28.067319277922312 mins
```

- The time complexity is upper-bounded by: $O(nd^2 + d^3)$
 - n = Number of samples
 - d = Number of variables

(may need some background CS knowledge to understand this)

[Latent Space Visualisation: PCA, t-SNE, UMAP | Deep Learning Animated](#)

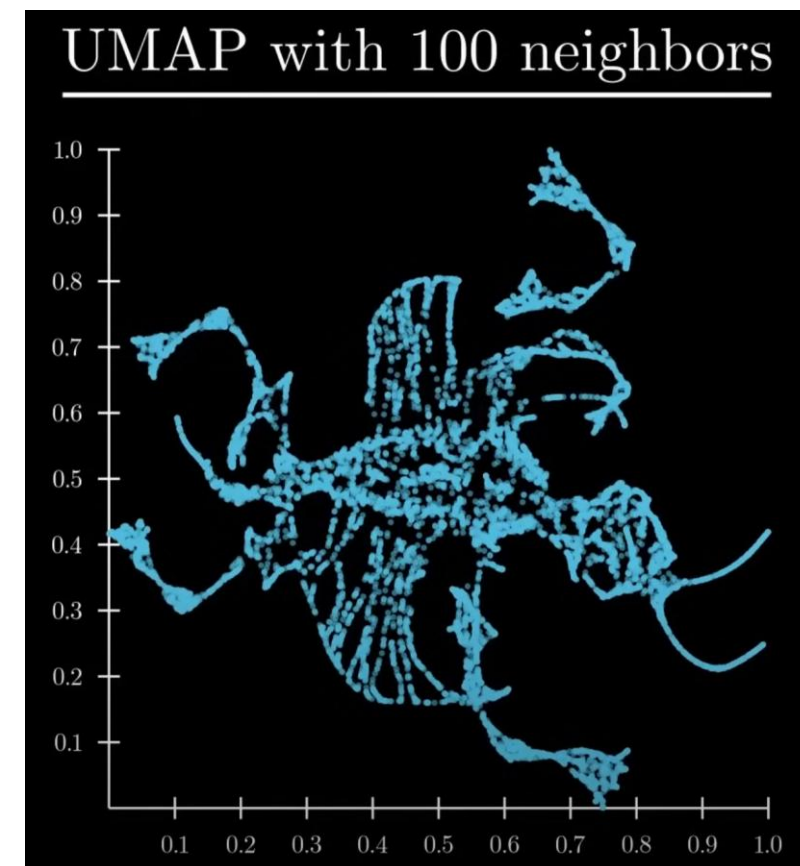
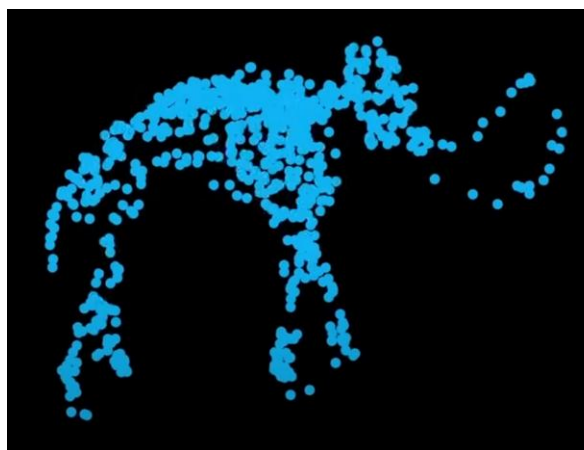
UMAP & Autoencoder Neural Network

- Future directions to explore!

UMAP (Uniform Manifold Approximation and Projection)

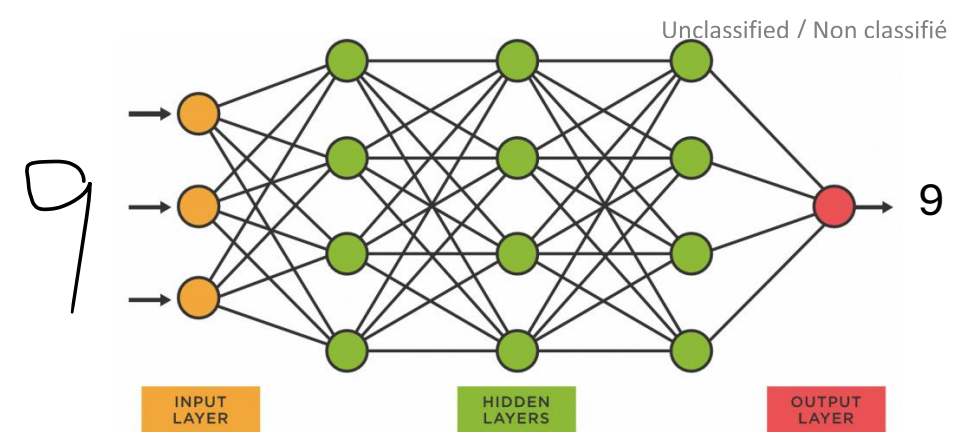
- Unsupervised dimension reduction technique!
 - Improvement on PCA: can be used on non-linear data!
 - Similar to t-SNE, but faster!
- Goal: Find a lower dimensional projection that has the closest possible equivalent “fuzzy topological” structure
- Method:
 - Use k-nearest neighbours to form a lower dimensional projection (k = hyperparameter)
 - Use loss function to minimize the difference between the high dimensional and low dimensional data

Original 3D Data



Notice how parts of the mammoth in 3D are also close in the 2D

Autoencoder Neural Network

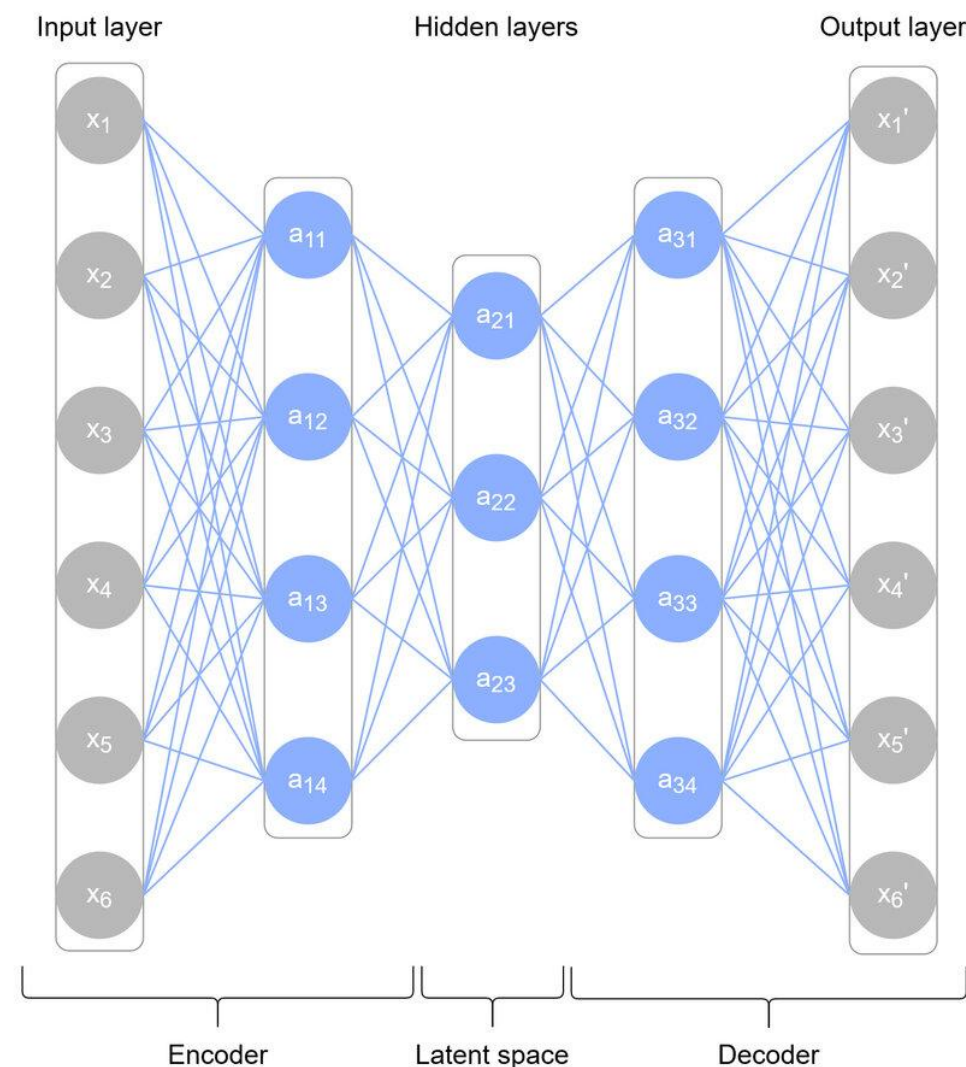


Top Neural Network Architectures For Machine Learning Researchers - MarkTechPost

- Consider watching this video as an introduction to neural networks: [But what is a neural network? | Deep learning chapter 1](#)
- Autoencoders are a type of neural network that
 1. Encodes – compresses input data to its most important features (dimension reduction like in PCA but also captures non-linear relations in data!)
 2. Decodes – rebuilds the original data from our compressed data.
- Main Uses: Feature Extraction, Data Compression, Anomaly Detection (and even generative tasks like image generation!)
- Autoencoders are an example of unsupervised learning, since they are trained to recreate the *input* data given to it (like PCA!)
 - Hence “auto” since the neural network is learning by itself!

Autoencoder: How does it Work?

- **Encoder:**
 - Takes the input layer and goes through multiple layers to reduce dimensions
 - Each consecutive layer usually has less “nodes/neurons”, thus compressing the data
- **Bottleneck/“Code”:**
 - Finds the “latent space” – “hidden/random variables that are fundamentally important”
 - Goal: Find least number of important features (i.e. dimensions) while still being able to effectively rebuild the data
- **Decoder:**
 - Has consecutively more nodes to decompress the data and reconstruct what we originally had
 - Reconstruction error: The difference between the reconstructed data and the original.



Autoencoder: Great Links!

- I used these links to help support my explanations, and they're pretty helpful resources! (read in given order!)
 - [What Is an Autoencoder? | IBM](#)
 - [Autoencoders in Machine Learning – GeeksforGeeks](#)
- A bit more extensions (definitely harder!)
 - Highly recommended! This is a really great visual and clear explanation on neural networks, but a very long playlist: [Neural Networks](#)
 - [Intro to Autoencoders | TensorFlow Core](#) (Python example code!)
 - Chapter 14 mentioned in the previous link – likely requires a heavy ML and statistics background: deeplearningbook.org/contents/autoencoders.html

Questions?

Contact me at: j23qiu@uwaterloo.ca